

PokeBin

Team 3

Tyler Usrey

Lucas Lazaroiu

Frankie Snedecor

Aidan Rodriguez

Software Design Specification & Project Delivery Report

Version: 1

Date: (11/9/2025)

Table of Contents

1 Introduction.....	5
1.1 Goals and objectives.....	5
1.2 Statement of system scope.....	5
1.3 Reference Material.....	6
1.4 Definitions and Acronyms.....	6
2 Architectural design.....	6
2.1 System Architecture.....	6
2.2 Design Rational.....	6
3 Key Functionality design.....	7
3.1 Collection Managing.....	7
3.1.1 Collection managing use case.....	7
3.1.2 Processing sequence for Collection Managing.....	9
3.1.3 Structural Design for Collection Managing.....	11
3.1.4 Key Activities.....	12
3.1.5 Software Interface to other components.....	13
3.2 LogOn action.....	13
3.2.1 LogOn use case.....	13
3.2.2 Processing sequence for Logon.....	15
3.2.3 Structural Design for LogOn.....	15
3.2.4 Key Activities.....	16
3.2.5 Software Interface to other components.....	16
3.3 Scan Card.....	16
3.3.1 Scan Card use Case.....	16
3.3.2 Processing sequence for Scan Card.....	17
3.3.3 Structural Design for Scan Card.....	18
3.3.4 Key Activities.....	18
3.3.5 Software Interface to other components.....	18
3.4 Add to Collection.....	18
3.4.1 Add to collection use case.....	18
3.4.2 Processing sequence for Add to collection.....	20
3.4.3 Structural Design for add to collection.....	20
3.4.4 Key Activities.....	21
3.4.5 Software Interface to other components.....	21
3.5 Delete Card.....	21
3.5.1 Delete Card.....	21
3.5.2 Processing sequence for Delete card.....	22
3.5.3 Structural Design for Delete card.....	22
3.5.4 Key Activities.....	23

3.5.5 Software Interface to other components.....	23
3.6 Calculate Total.....	23
3.6.1 Calculate Total use case.....	23
3.6.2 Processing sequence for Calculate Total.....	25
3.6.3 Structural Design for Calculate Total.....	26
3.6.4 Key Activities.....	26
3.6.5 Software Interface to other components.....	26
3.7 Search card.....	26
3.7.1 Search card Use case.....	26
3.7.2 Processing sequence for search card.....	27
3.7.3 Structural Design for search card.....	27
3.7.4 Key Activities.....	28
3.7.5 Software Interface to other components.....	28
4 User interface design.....	28
4.1 Interface design rules.....	28
4.2 Description of the user interface.....	28
4.2.1 Login Page.....	28
4.2.2 Main Menu Page.....	29
4.2.3 Manual Input Page.....	30
4.2.4 Scan Page.....	31
4.2.5 Collection Page.....	32
5 Restrictions, limitations, and constraints.....	33
6 Testing Issues (SLO #2.v).....	33
6.1 Types of tests.....	33
6.2 List of Test Cases.....	33
6.3 Test Coverage.....	34
7 Appendices.....	35
7.1 Packaging and installation issues.....	35
7.2 User Manual.....	35
7.3 Open Issues.....	37
7.4 Lessons Learned.....	37
7.4.1 Project Management & Task Allocations (SLO #2.i).....	37
7.4.2 Implementation (SLO #2.iv).....	38
7.4.3 Design Patterns.....	39
7.4.4 Team Communications.....	39
7.4.4 Technologies Practiced (SLO #7).....	40
7.4.6 Software Security Practice (SLO #4.iii).....	40
7.4.5 Desirable Changes.....	40
7.4.6 Challenges Faced.....	41

1 Introduction

Card collecting is a huge market, today many people spend lots of money and collect vast amounts of cards, but these cards aren't cheap. You can spend money on packs that give you a chance at getting the card you want, or you can spend money on single cards. A problem people face or don't realize is how much they have spent compared to how much they have earned. You could spend hundreds of dollars on packs, finally pull the chase card, but you don't realize that you spent almost double on packs compared to the price of the card. Our app helps solve this problem as we aim to make a quick, easy and efficient way for people to track the money they've spent and earned, as it becomes very easy to forget how much you have spent. This system can help users make better financial decisions for themselves and allow them to be smart with their spendings.

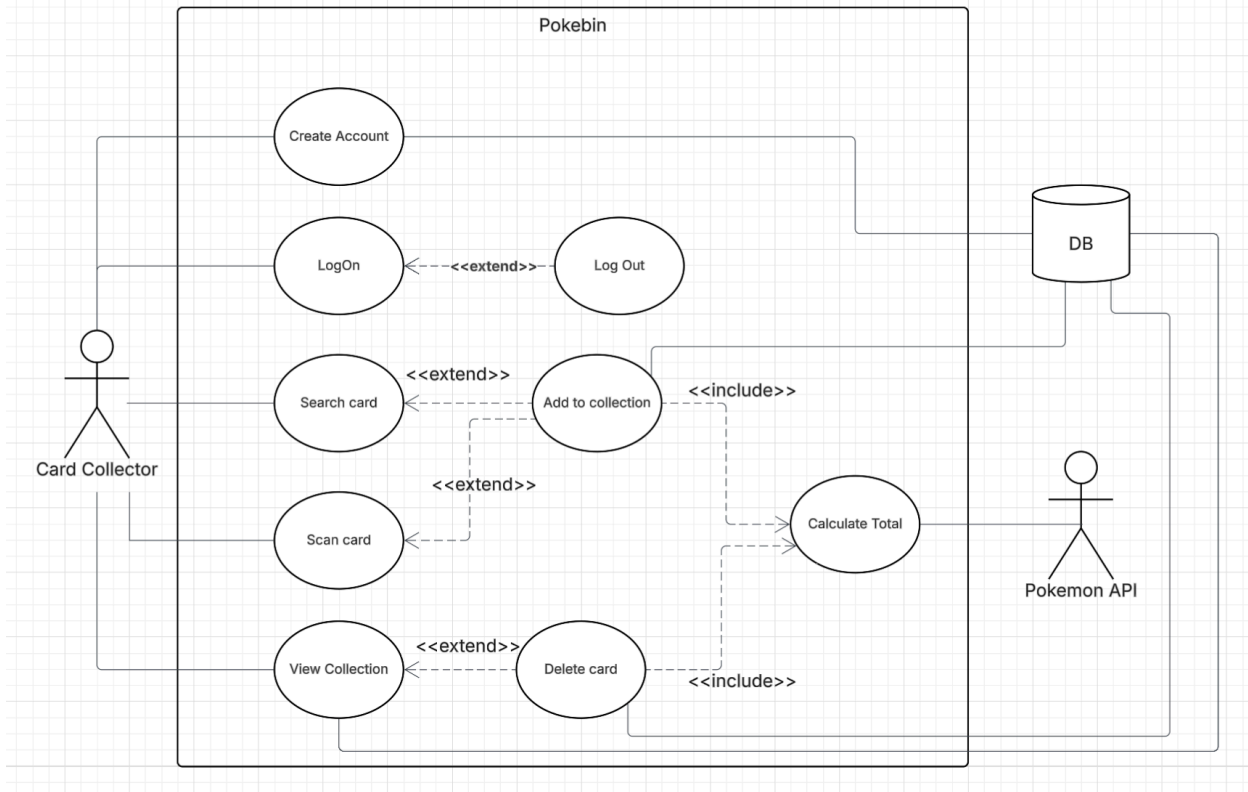
1.1 Goals and objectives

To create an application to keep track and collect cards that the user opens or has in their collection. The user should be able to see their collections total value, as well as all of their cards in the collection.

1.2 Statement of system scope

The software will retrieve card data from the TCGDex API and store user card information inside of a mySQL database. The user will be able to access this database frequently as they should be able to see their card collection.

UML use case diagrams



everything organized and separate. Each layer specifically separates tasks that do NOT depend on each other. The facade method allows us to have one simple entry point into our functions. Without our facade we would need more functions and interfaces providing potential errors, having everything interact with our one interface keeps it visually well structured and organized. The DAO layer that we implemented allows a nice and easy way to communicate to the database and the API calls that are being created. If we implemented a Client Server it may have worked in the short term run, as in it would work if we wanted to just remain at pokemon cards, but if we wanted to grow the application into something more and scan other cards other than pokemon the layered architecture style provides us a long term benefit that has great scalability overall.

3 Key Functionality design

3.1 Collection Managing

3.1.1 Collection managing use case

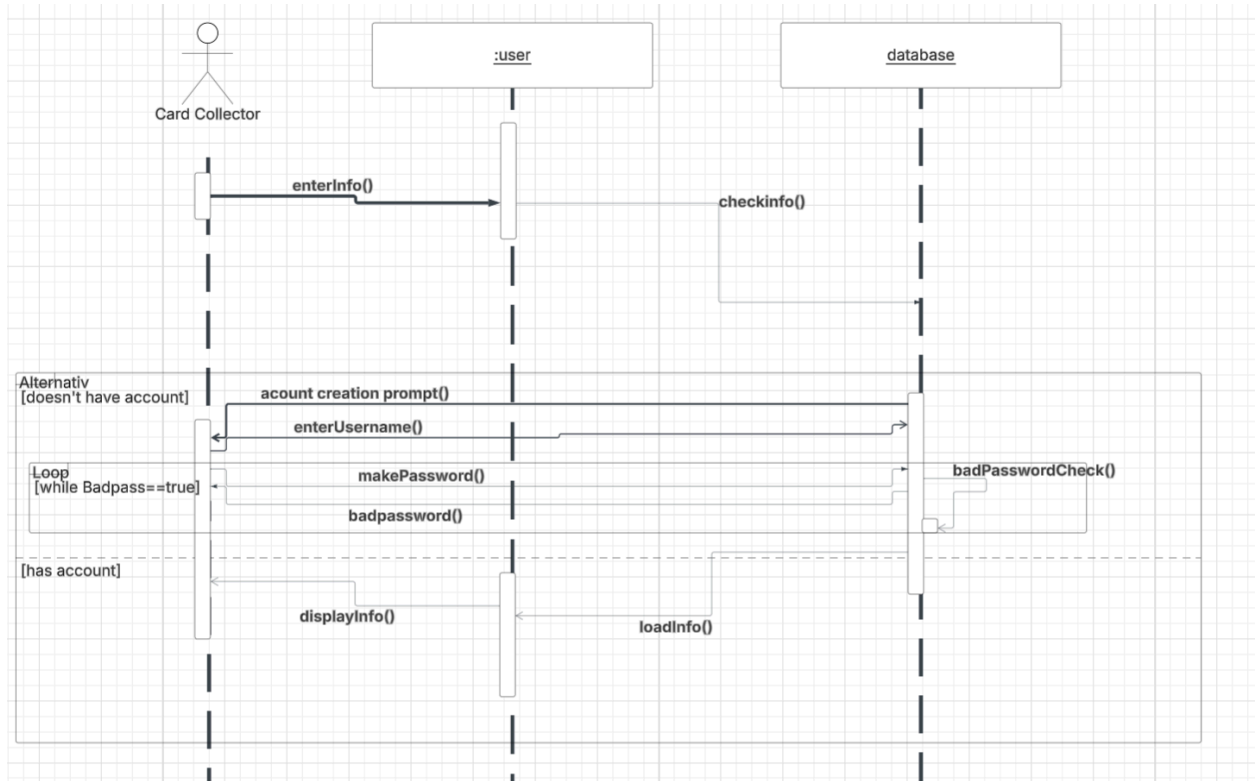
Use case	A collector managing their collection
Goal in context:	Card collector using the app
Scope	App system
Level	Summary
Primary Actor	Card Collector 🧑🏻
Preconditions	App not open 🧑🏻
Minimal Guarantee	Still not open 😊
Success Guarantee	Collection updated
Trigger	The card collector starts application
Success Scenario	action
1	Collector logs-in using <<LogOn>>
2	Validate password
3	Collection initializes
4	Collector Scans card with <<Scan card>>
5	Collector adds card with <<Add to collection>>
6	Collection updates
7	Total updates <<Calculate total>>
8	Collector views collection with <<View collection>>

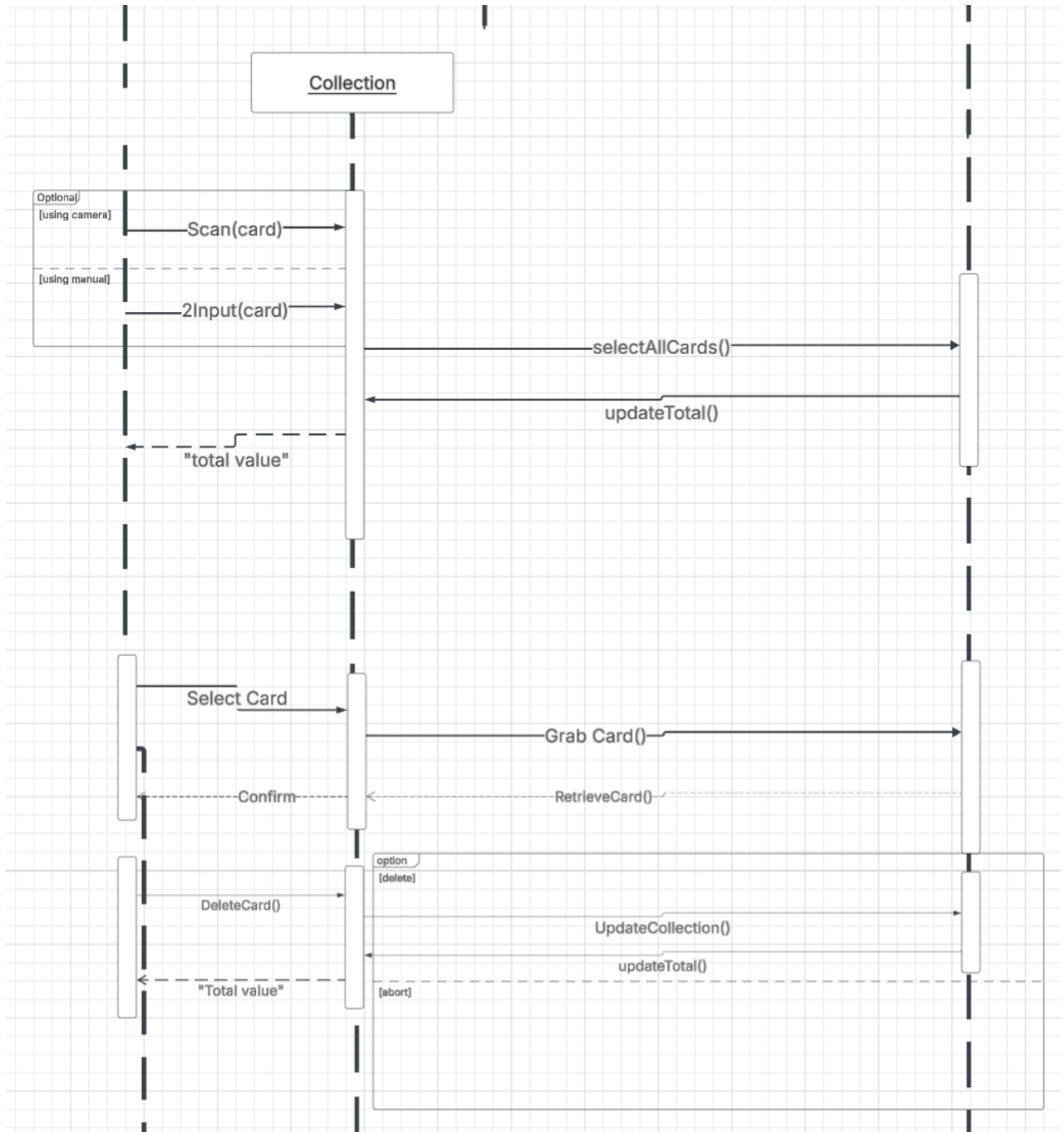
9	Collector deletes card with <<Delete card>>
10	Collection updates
11	Total updates <<Calculate total>>
12	Collector views collection with <<View collection>>
13	Collector logs out with <<Log out>>
14	Collector terminates app
Extension step	Branching actions
2a	Collector enters incorrect password
	A.1 Validates that password is incorrect
	A.2 Prompts system warning
	A.3 Go back to main step 1
4a	Camera doesn't work and can't read card
	A .1 Card Collector manually searches card with <<Search card>>
	A.2 Continue on from main step 5
9a	Card doesn't exist to delete
	A.1 Card input doesn't match with card in database
	A.2 Prompt card error
	A.3 Go back to main step 8

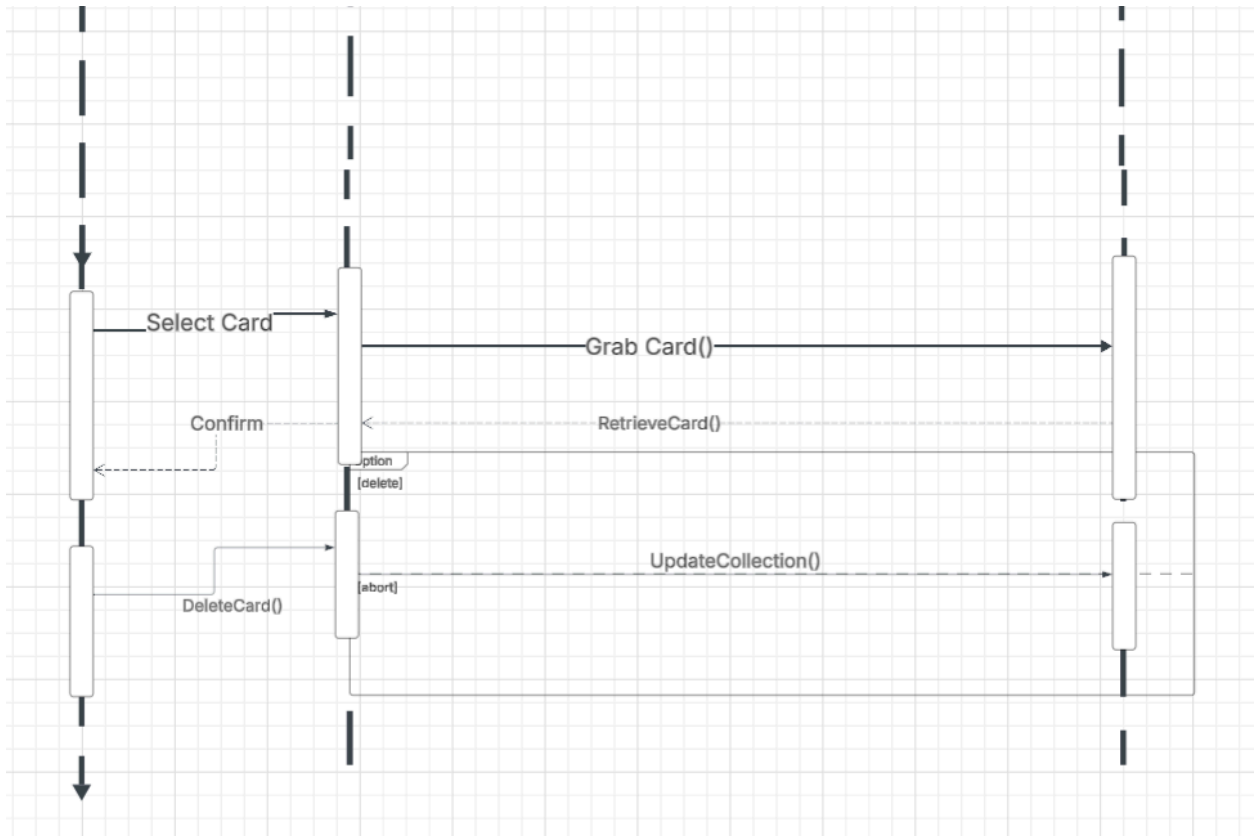
The collector will first open the app and attempt to login. Depending on if the username and password are correct they will either be prompted to reenter the information or they will continue on to the next step. They will then be able to choose the button to access the collection. Then in the collection they will be able to modify the collection and view the total value of the cards there. Then they press logout and the app will log them out and close. There by ending the use case.

3.1.2 Processing sequence for Collection Managing

[UML sequence diagrams]

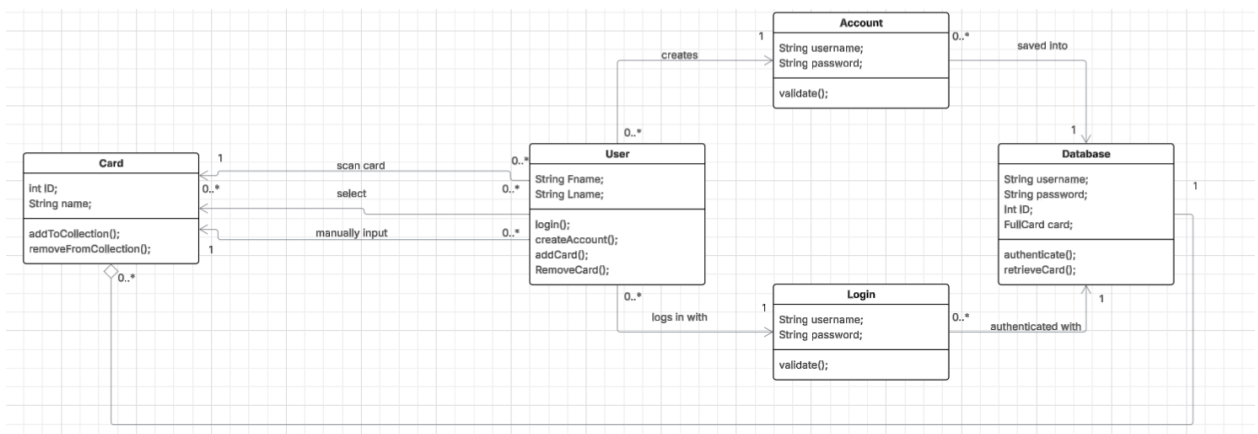




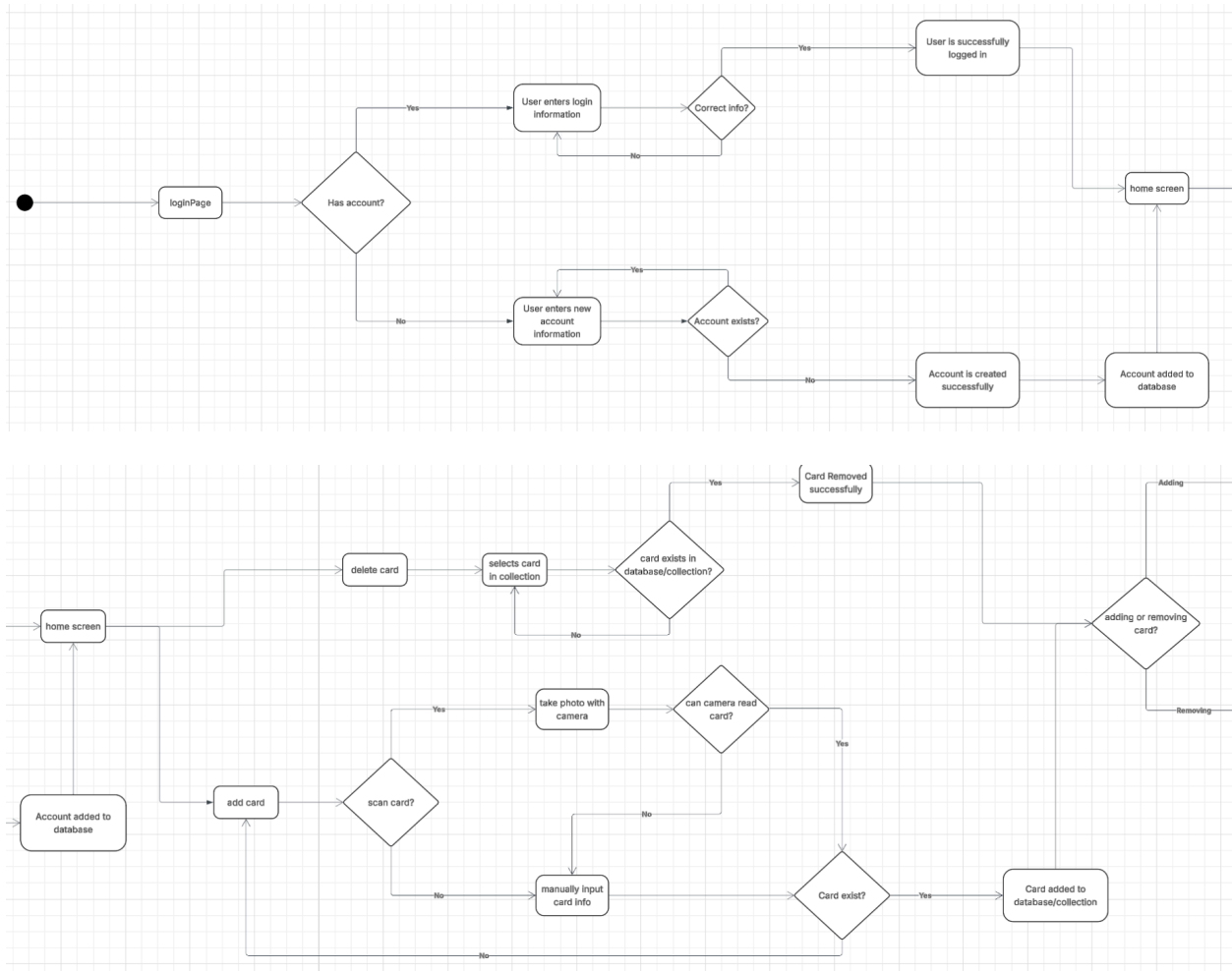


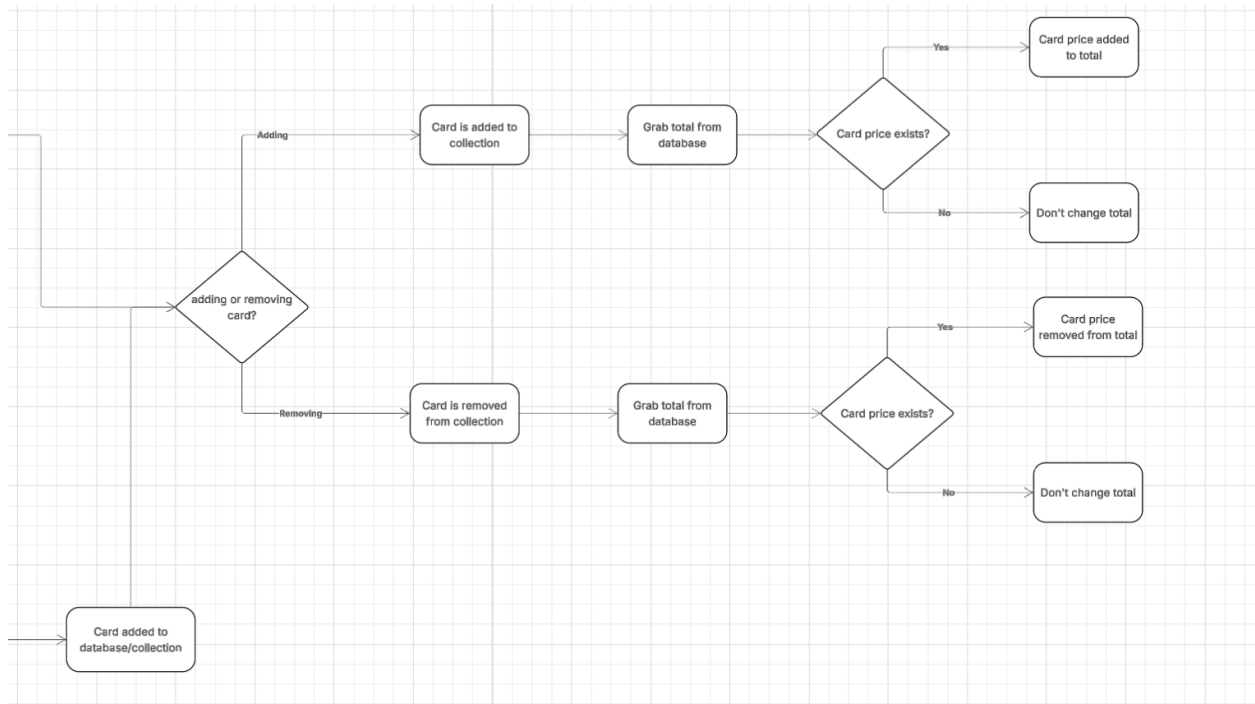
3.1.3 Structural Design for Collection Managing

[UML class diagrams]



3.1.4 Key Activities





3.1.5 Software Interface to other components

3.2 LogOn action

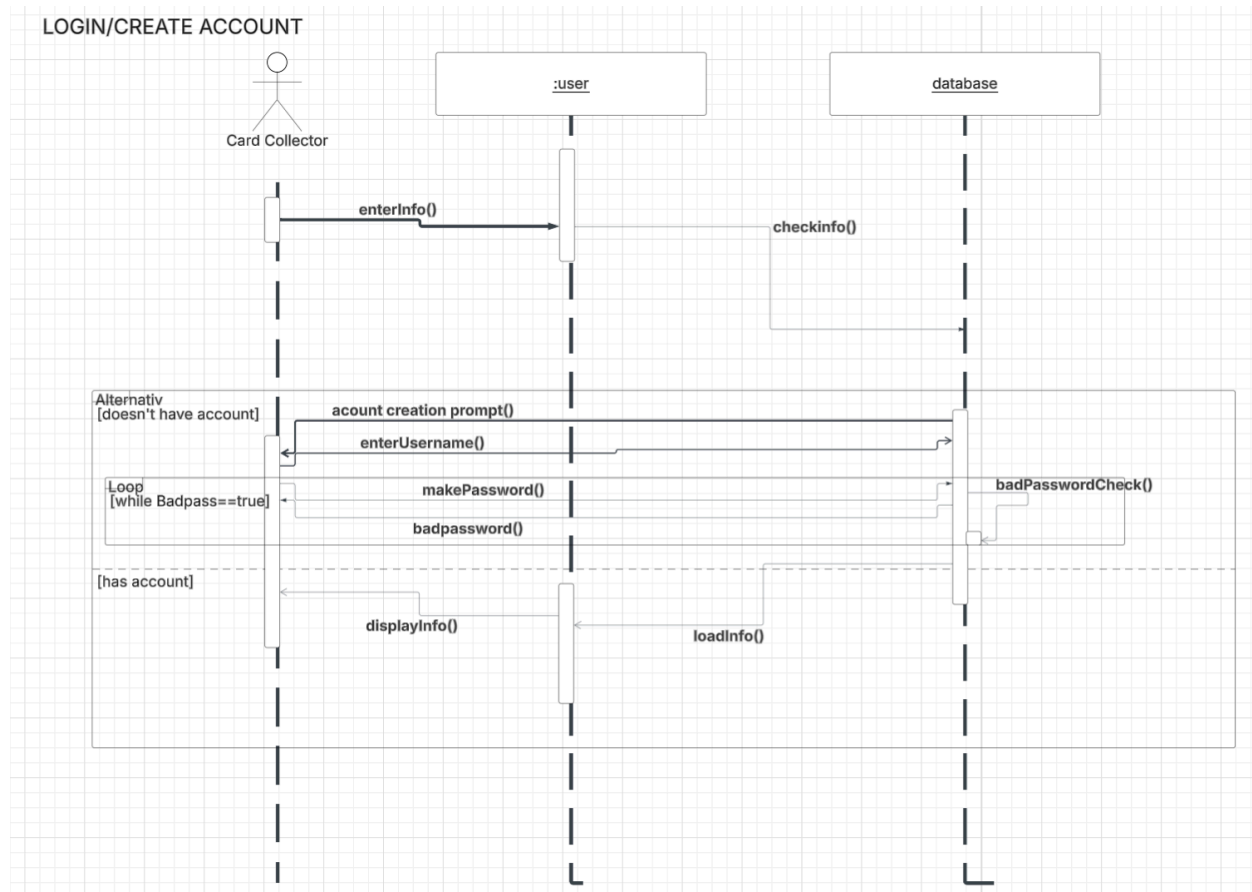
3.2.1 LogOn use case

Use case	LogOn
Goal in context:	A collector needs to login in order to have access.
Scope	PokeBin
Level	primary
Primary Actor	Card Collector
Preconditions	Started and ready to login
Minimal Guarantee	Card Collector fails login
Success Guarantee	Into the app
Trigger	The card collector starts application
Success Scenario	Application started
1	Collector inputs username

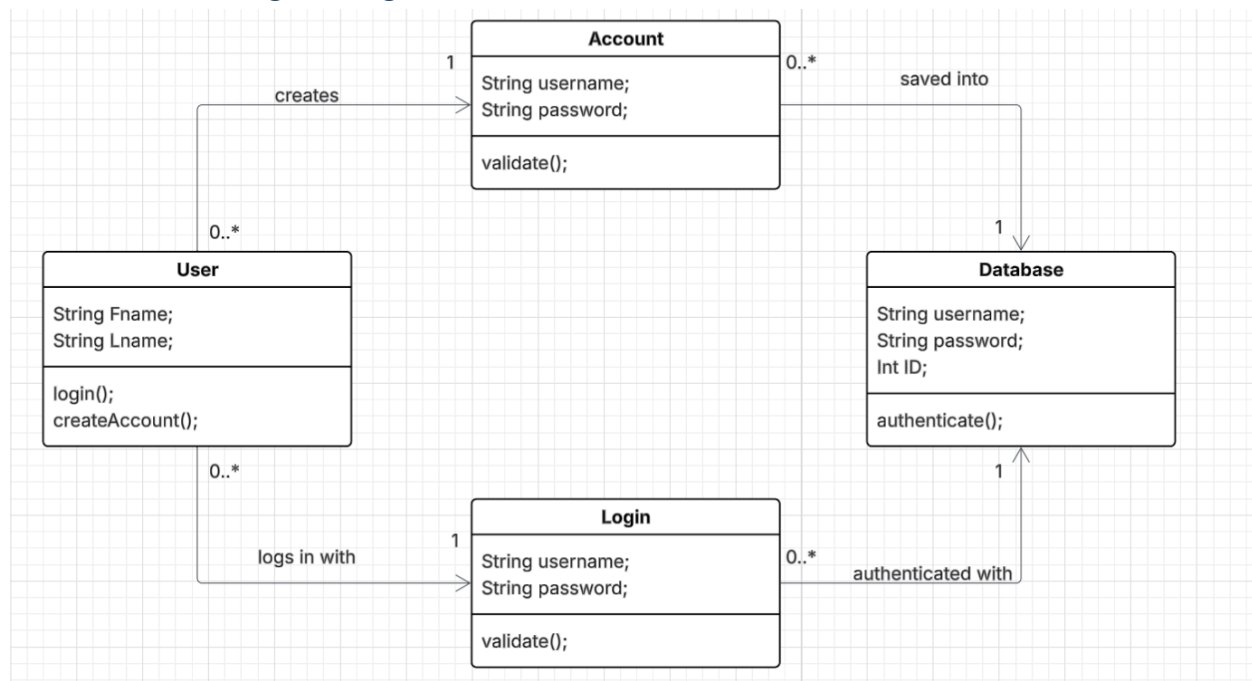
2	Collector inputs password
3	Correct password validated
4	Log into collection
Extension step	Branching actions
3a	A.1 Validates that password is incorrect
	A.2 Prompts system warning
	A.3 Continue as long as password is incorrect up to 10 times

Log on use case. The prior conditions for this must be the app is open and the collector is not already logged in. Then the sequence will be as follows. First the username must be entered and then the password is entered. Next the password is validated. This means it is sent from the Ui layer to the application layer all the way to the Database layer in order to check and see if the user name and password are correct. They are correct and this message is sent back to the application layer that then will perform the requisite tasks to set up the main menu. Then we have an extension that says if the password is wrong tell the user and allow for re-input up to 10 more times.

3.2.2 Processing sequence for Logon

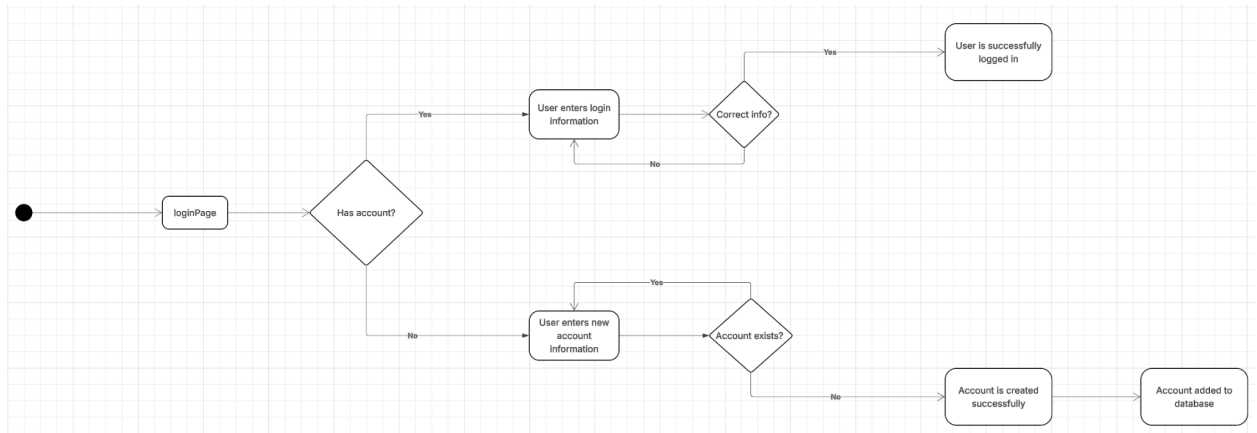


3.2.3 Structural Design for LogOn



3.2.4 Key Activities

[UML activity diagrams]



3.2.5 Software Interface to other components

N/A

3.3 Scan Card

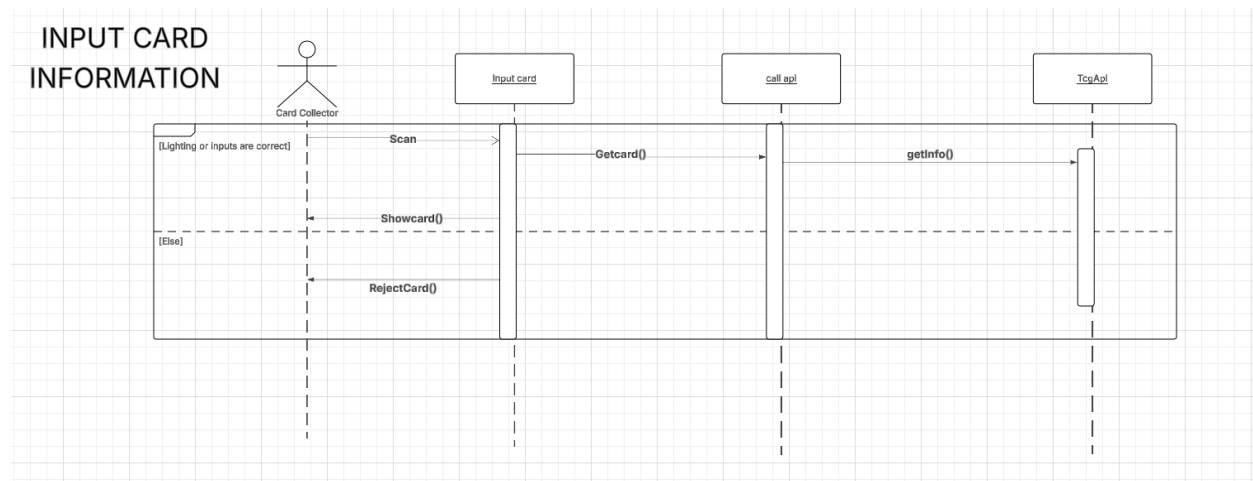
3.3.1 Scan Card use Case

Use case	Scan card
Goal in context:	Collector can scan their card to find it
Scope	Pokebin
Level	Action
Primary Actor	Card Collector 🕵️
Preconditions	Logged in, Camera on
Minimal Guarantee	Card not added
Success Guarantee	Card added to collection
Trigger	Collector presses scan card button
Success Scenario	action
1	Collector presses scan card button
2	Card is validated by name and number via camera
3	Card is found
4	Card is added to collection

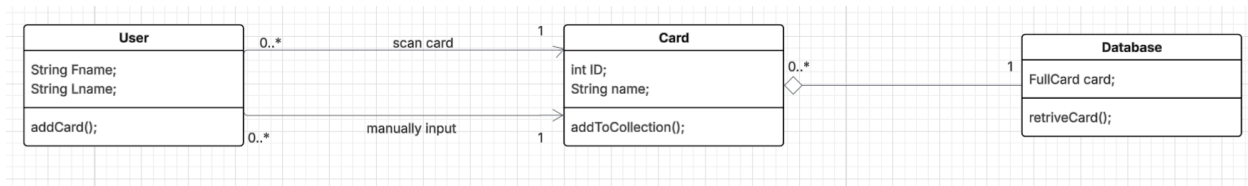
5	Collection is updated
Extension step	Branching actions
3a	A.1 could not find identifier for the card
	A.2 Prompts system warning pertaining to lighting
3b	B.1 Could not find identifier for card
	B.2 prompts system warning pertaining to clarity
4a	A.1 Could not find card
	A.2 tell user to use the search function.

The Prior conditions for this are that the Collector is logged into the app and has a camera on the device that is on and that we have access to. The steps to success for this use case are, first the Collector presses the scan card button, second the app can locate and validate the unique identifier on the card and the pertinent information, this is done on the application layer. Next that information will be sent from the application layer to the database layer where we will send it to the TCGApi to get the price of said card. Then that info is sent back to the application layer. Then that layer sends a request back to the database layer to then get the Collectors collection and add the card. And then the collection is updated. We have an extension step that states two warning messages if the camera could not read the card. And another to give a warning if the card could not be found with the scan function to input the info into the application using the search function.

3.3.2 Processing sequence for Scan Card

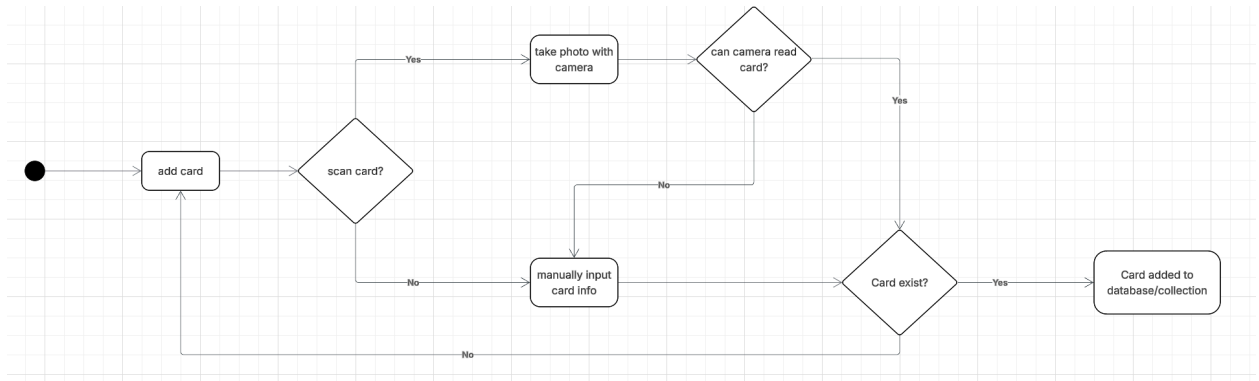


3.3.3 Structural Design for Scan Card



3.3.4 Key Activities

[UML activity diagrams]



3.3.5 Software Interface to other components

3.4 Add to Collection

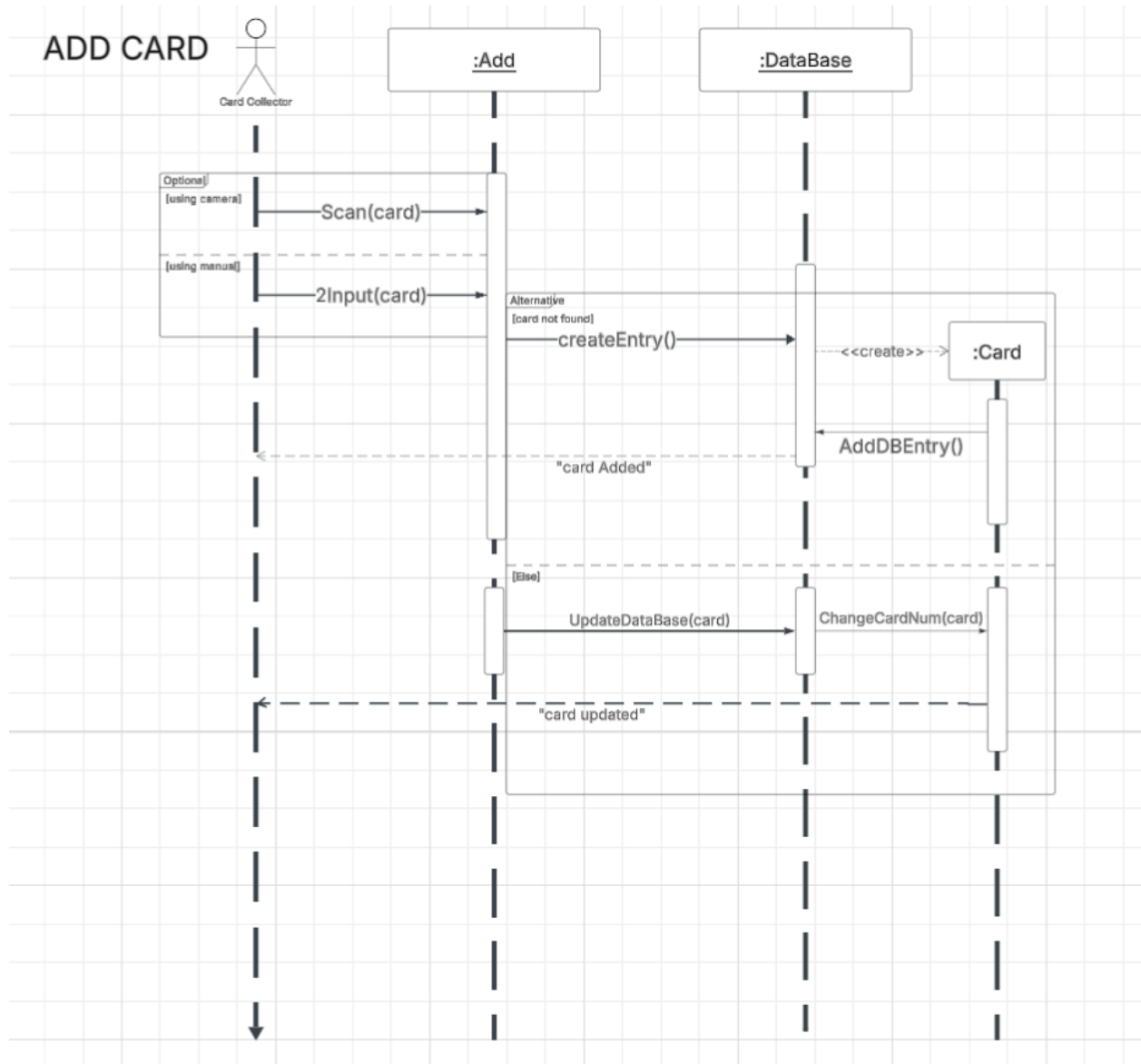
3.4.1 Add to collection use case

Use case	Add to collection
Goal in context:	Collector can add their card to their collection
Scope	Pokebin
Level	subfunction
Primary Actor	Card Collector 🧑🏻
Preconditions	Card must be searched or scanned
Minimal Guarantee	Card not added
Success Guarantee	Card added to collection
Trigger	Collector presses action button
Success Scenario	action
1	Collector Presses add card button
2	Card info is saved to a holder
3	Holder is sent to collection

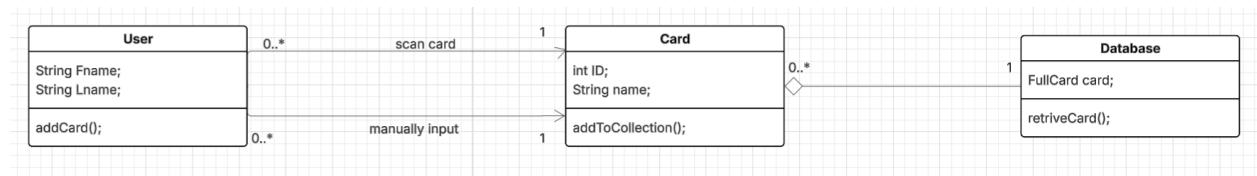
4	Collection receives holder
Extension step	Branching actions
4a	A.1 did not receive holder
	A.2 system warning stating failed to add

The preconditions for the subfunction of adding to the collection are as follows: the user has done the search or scan card function, and said card has been found in the TCGapi. First the collector presses the add card button, Next we take the info returned to the main screen and insert it into a holding object. This object will be used to transfer the pertinent info into the Database. Next the application layer sends the card object to the database which tries to add the info. It succeeds and returns a success statement. There is an extension step that if the collection fails to add the info the application should send a message to the ui to put a notification to try again.

3.4.2 Processing sequence for Add to collection

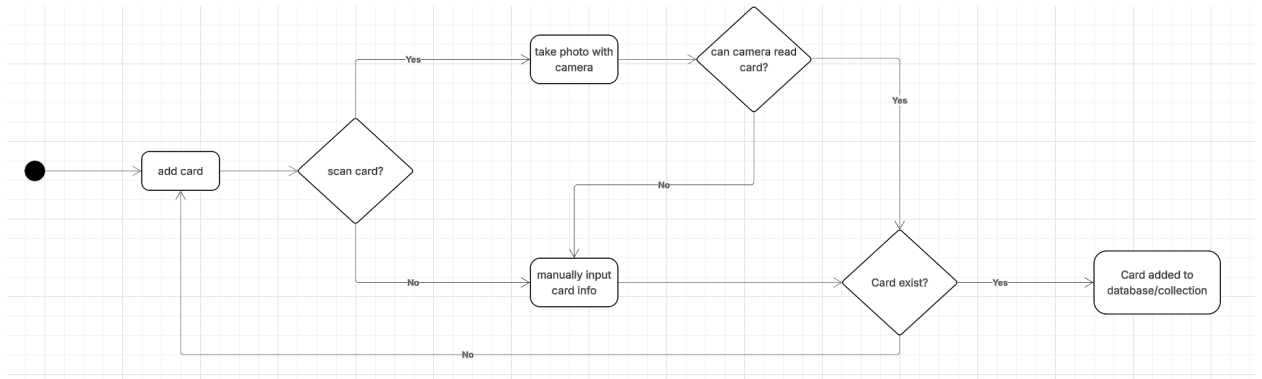


3.4.3 Structural Design for add to collection



3.4.4 Key Activities

[UML activity diagrams]



3.4.5 Software Interface to other components

3.5 Delete Card

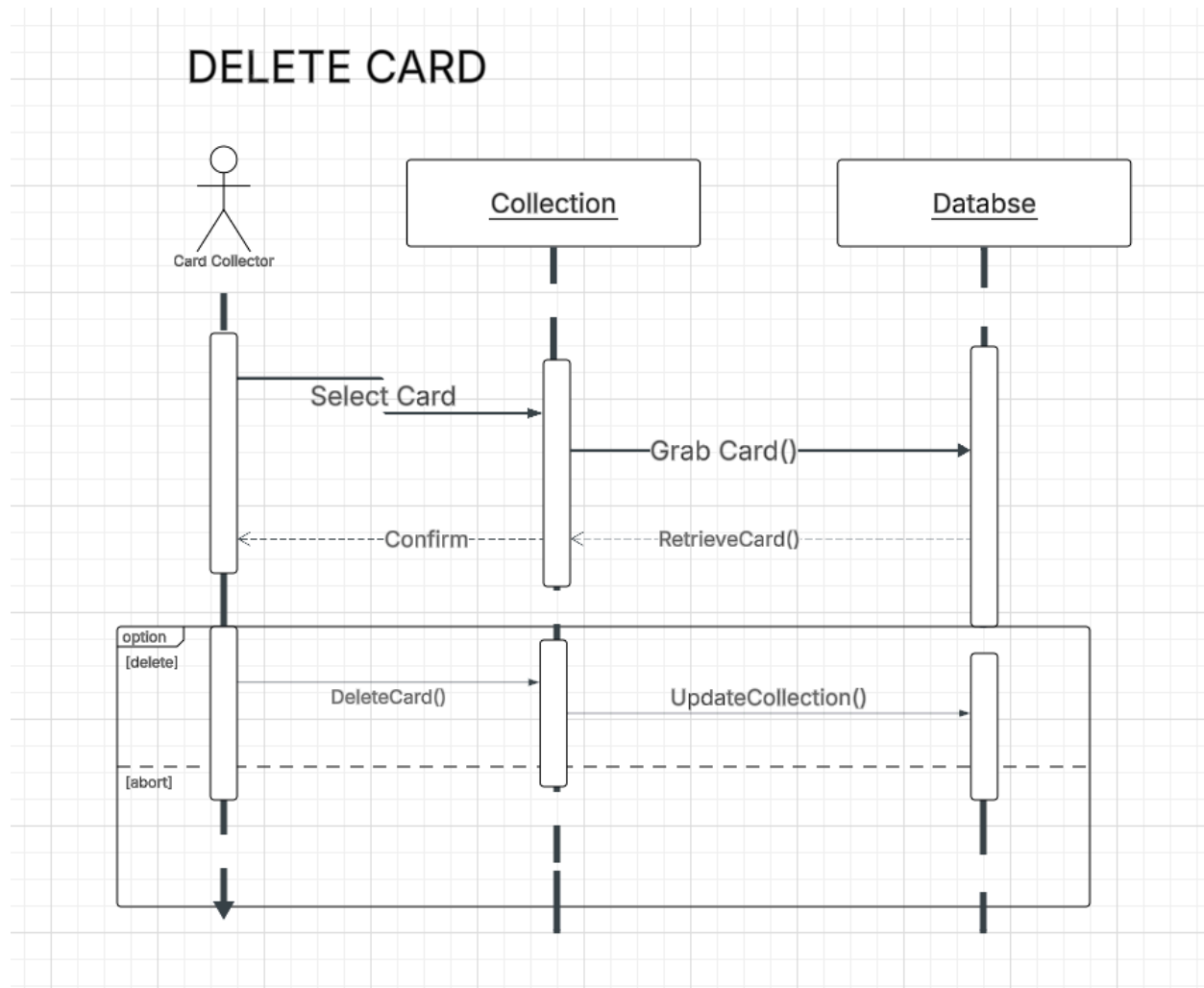
3.5.1 Delete Card

Use case	Delete card
Goal in context:	Collector can delete their card from their collection
Scope	Pokebin
Level	subfunction
Primary Actor	Card Collector 🧑
Preconditions	View collection
Minimal Guarantee	Card not in collection
Success Guarantee	Card removed from collection
Trigger	Collector presses action button
Success Scenario	action
1	Collector Presses remove card button
2	Card info is saved to holder
3	Holder is sent to collection
4	Collection finds card matching holder
5	Card is removed from collection
Extension step	Branching actions

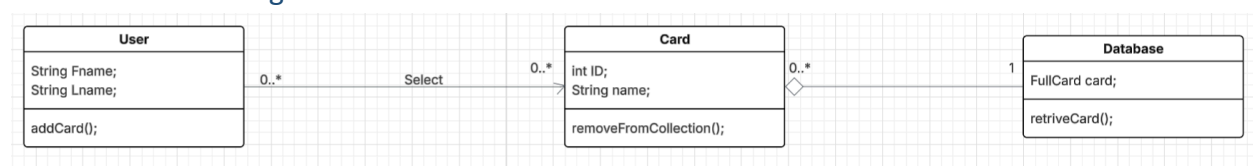
4a	A.1 did not receive holder
	A.2 system warning stating failed to remove

The preconditions for delete card are that they are in their collection and that there is a card to delete. The steps for this sequence are the collector is on the card and then presses the remove card button. The pertinent info is sent to the collection through a DAO to be deleted. The collection looks for the card and removes it and sends back a success message. The extension is if the database does not receive the information it shall tell the application layer to send back an error message of “failed to remove”.

3.5.2 Processing sequence for Delete card

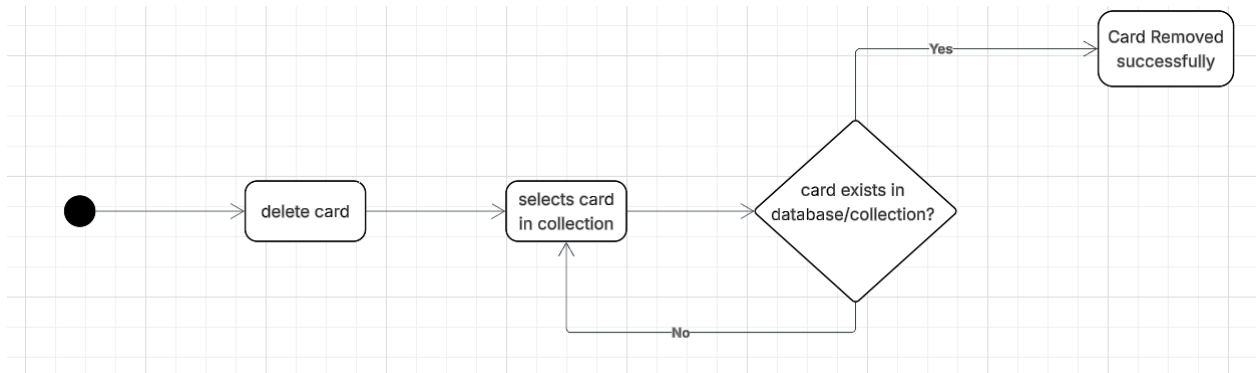


3.5.3 Structural Design for Delete card



3.5.4 Key Activities

[UML activity diagrams]



3.5.5 Software Interface to other components

3.6 Calculate Total

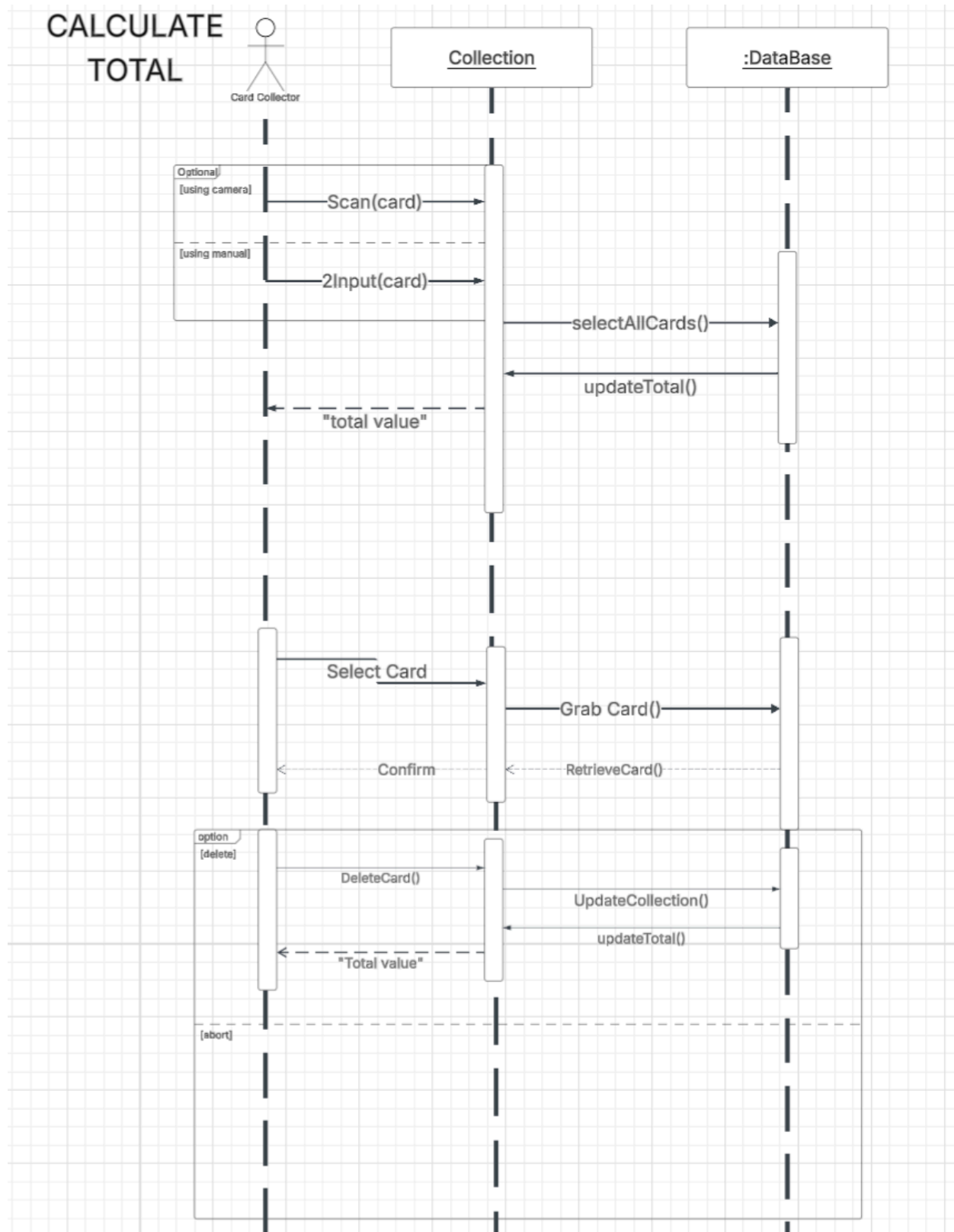
3.6.1 Calculate Total use case

Use case	Calculate Total
Goal in context:	Summation of card prices
Scope	Pokebin
Level	subfunction
Primary Actor	App
Preconditions	New card added to collection or deleted
Minimal Guarantee	Fails to load prices
Success Guarantee	Total of all card prices returned
Trigger	Collector adds new card to collection or deletes card from collection
Success Scenario	Total changed
1	Collector adds card to collection
2	App takes the active total of the card prices
3	App finds the price of the added card
4	App adds price to total
Extension step	Branching actions
3a	A.1 cannot find card price

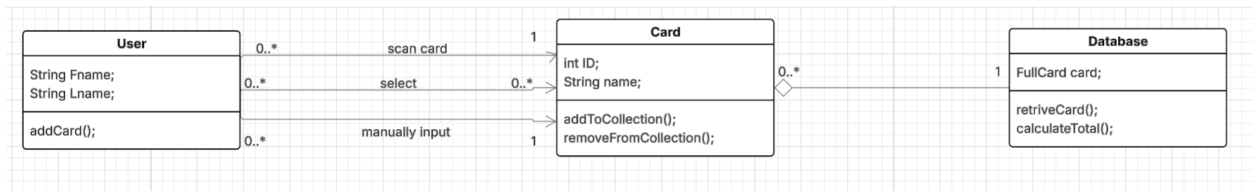
	A.2 system warning stating card cannot be found
	A.3 system goes back to step 1
1a	1 Collector removes card from collection
	2 App takes the active total of the card prices
	3 App finds the price of the removed card
	4 App removes price from total

The preconditions for this is that a card has been added or deleted from the collection. This will trigger the following sequence. The card was added, in to calculate the total, get the new card price, add the price to the total. The calculation of the total could be circumvented with adding collection total to the database. The extension step is concerned with if it can't find the card price. If that is true it resets and tells it to run again. And if the card is removed it follows the same logic except subtracting instead of adding to the total.

3.6.2 Processing sequence for Calculate Total

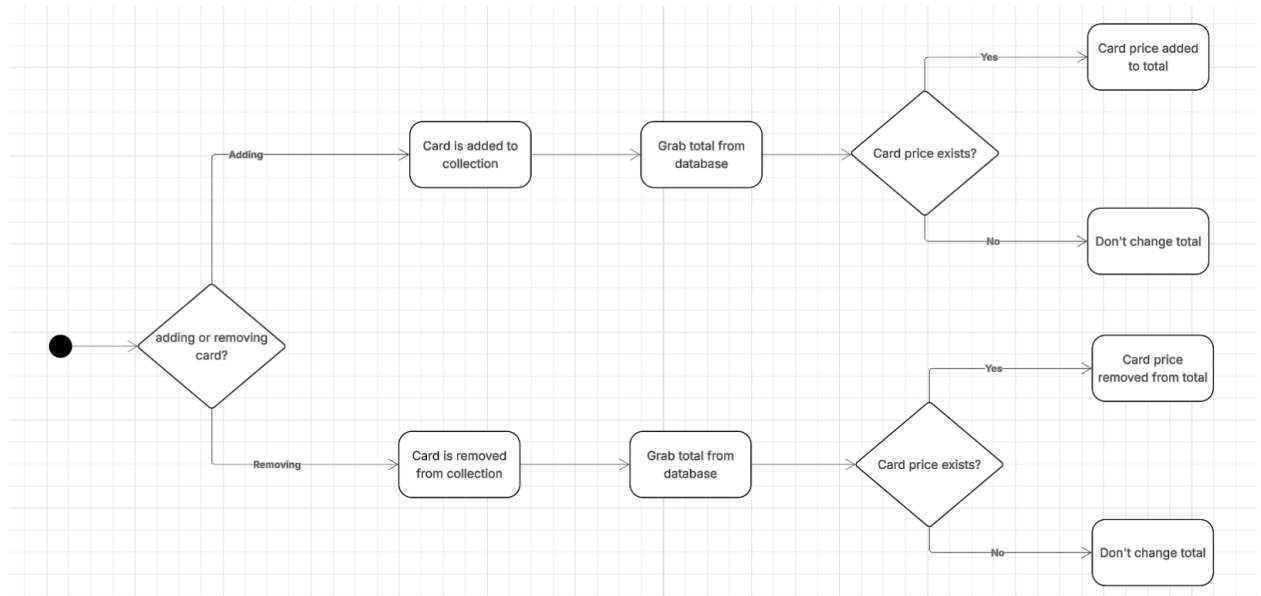


3.6.3 Structural Design for Calculate Total



3.6.4 Key Activities

[UML activity diagrams]



3.6.5 Software Interface to other components

3.7 Search card

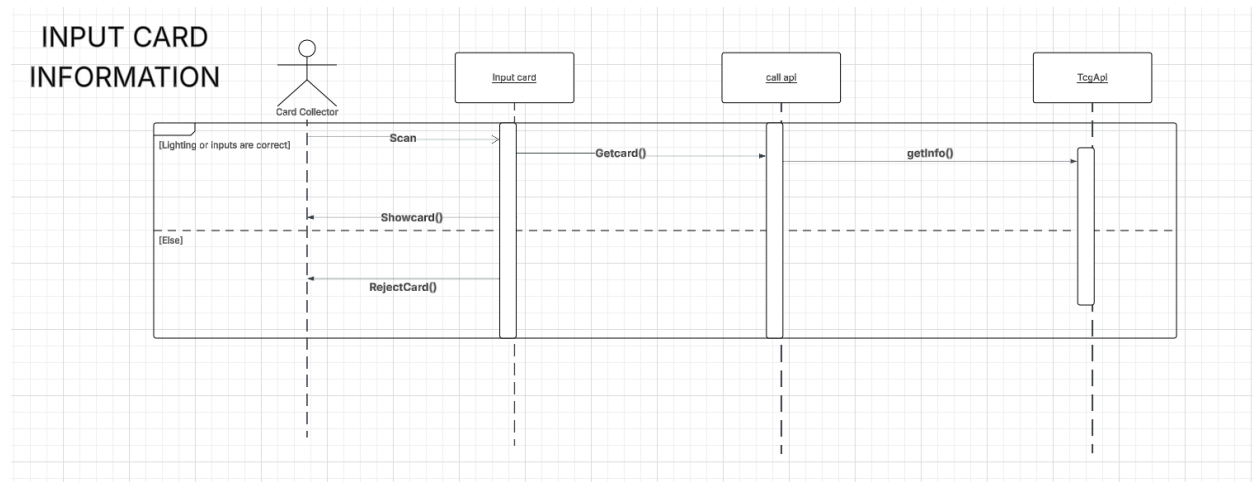
3.7.1 Search card Use case

Use case	Search card
Goal in context:	A collector manually enters the card information to search
Scope	PokeBin
Level	Primary
Primary Actor	Card Collector
Preconditions	Card Collector is logged, has card
Minimal Guarantee	Card not found
Success Guarantee	Card is added to collection
Trigger	The card collector selected search card button

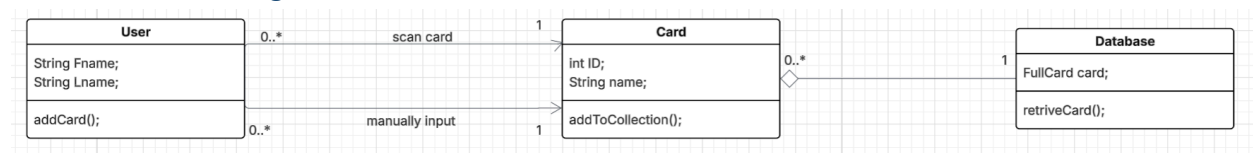
Success Scenario	action
1	Collector finds unique identifiers of card
2	Collector inputs unique identifiers
3	Successfully finds card
Extension step	Branching actions
2a	A.1 System does not return the card if card inputted incorrectly
	A.2 System Warning to double check input

The precondition for this action is that the collector is logged in. The sequence that we will follow is that the collector finds the identifiers for the card. Then the collector inputs said information into the ui. Then that info is sent to the application layer which then parses through and sets all the info up in to a holder object. This object is sent to the database layer which then sends it to the TCGapi. Then it takes the returned information and sends it back to the application. Then that info is sent to the ui. The extension for this process is that if the info is unable to be found due to miss inputted info a warning will pop up to reinput the card info.

3.7.2 Processing sequence for search card

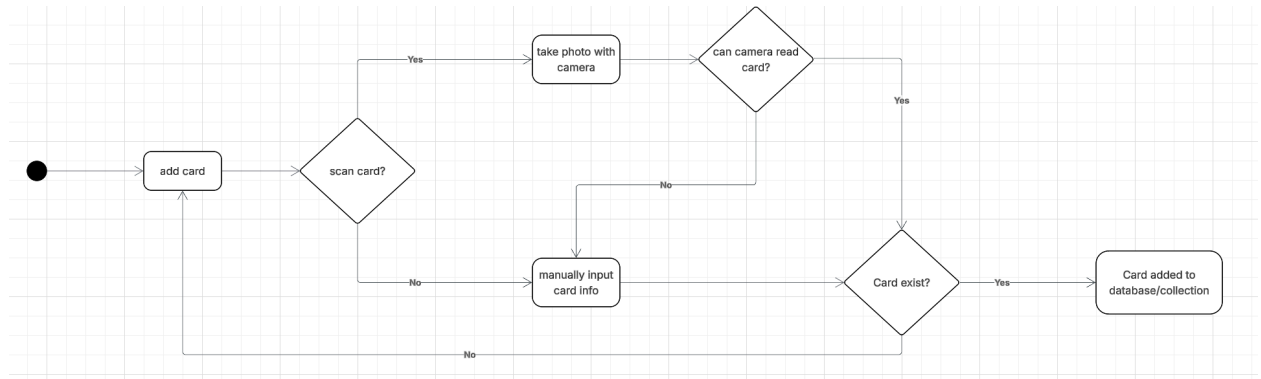


3.7.3 Structural Design for search card



3.7.4 Key Activities

[UML activity diagrams]



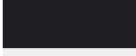
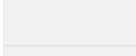
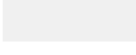
3.7.5 Software Interface to other components

4 User interface design

4.1 Interface design rules

The color scheme standard we went with was taken from the *Pokeball* from the game *Pokemon* for a clean and modern look. See Figure 4.1.

Colors in Palette

Color	Hex	RGB
	#ee1515	(238,21,21)
	#f00000	(240,0,0)
	#222224	(34,34,36)
	#f0f0f0	(240,240,240)
	#f0f0f0	(240,240,240)

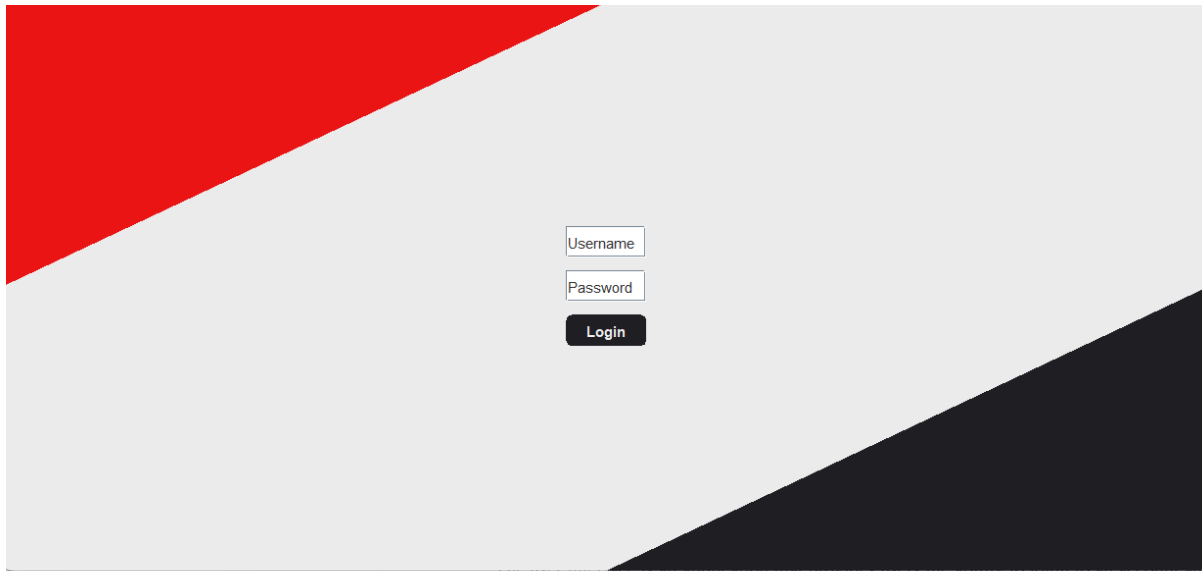
(fig 4.1)

4.2 Description of the user interface

The user interface is an application that provides users the ability to login to an account and the option to add cards, view cards, and delete cards from their own accounts.

4.2.1 Login Page

The login page allows the user to input their username and password, if the username and password exists inside the database, they can get into the application.



4.2.2.1 *Objects and Actions*

The only objects and actions of this page are the buttons to take the user to different pages and the change to their respective pages.

4.2.2 Main Menu Page

The main menu page is an interface that allows the user to navigate to the add cards to their collection via a manual input or a scan, check their collection, or log out of their account. The user is taken to the manual input page, scan page, collection page, and back to the login page, respectively.

4.2.2.1 Screen Images

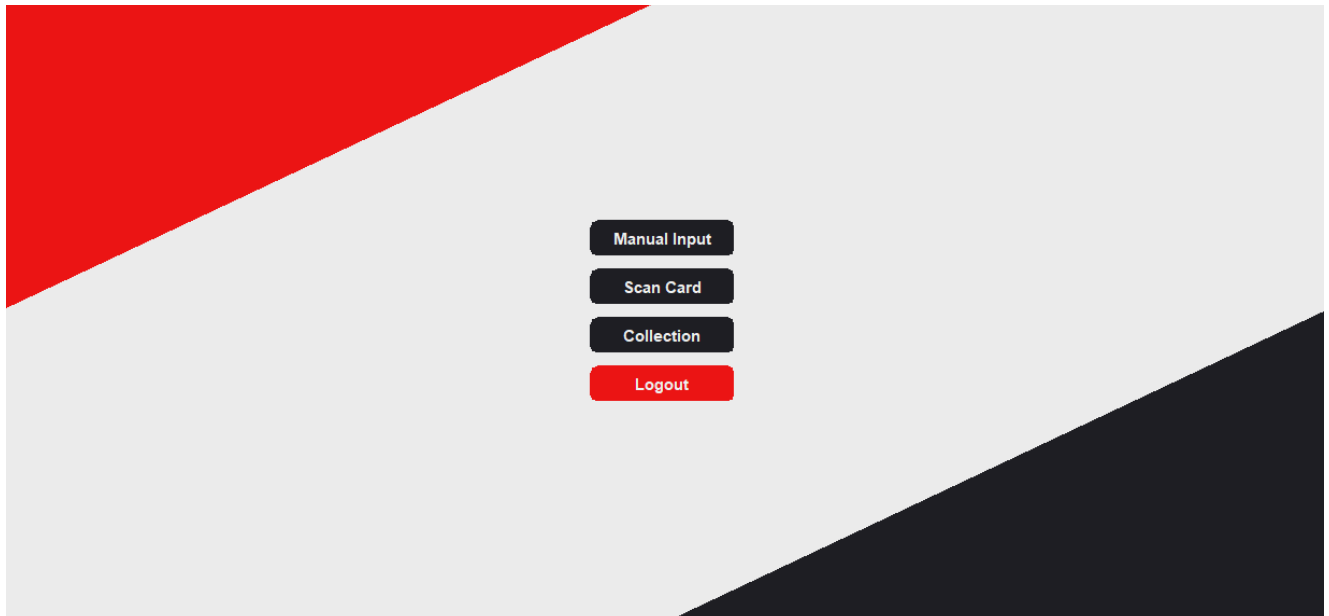


fig. (4.2.2)

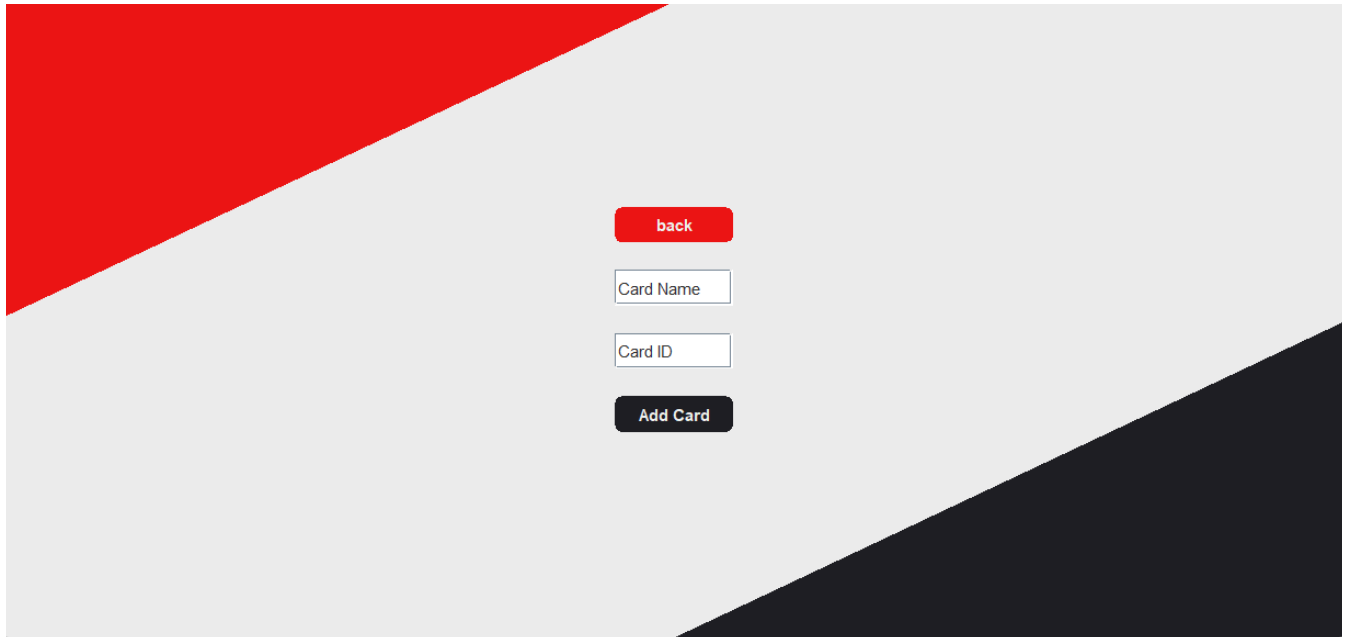
4.2.2.1 Objects and Actions

The only objects and actions of this page are the buttons to take the user to different pages and the change to their respective pages.

4.2.3 Manual Input Page

The Manual Input Page allows the user to input the name and Unique Pack ID of a card and add the card to their collection. It will return an error if the card does not exist.

4.2.2.1 Screen Images



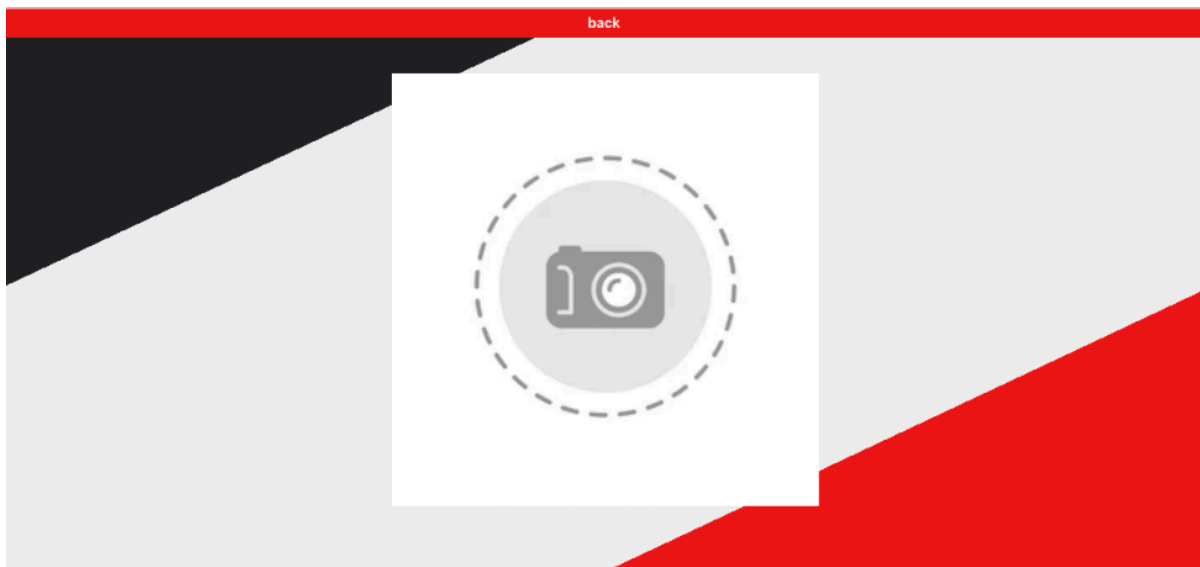
4.2.2.2 Objects and Actions

The objects of this page are the back button, card identifier inputs, and the add card button. The back button will bring you back to the main menu, the card identifier inputs take the name and Unique card identifier for a card, and the Add Card sends an API call to the TCGDex API to get the information of the card using the 2 inputs.

4.2.4 Scan Page

The scan page is a camera input from the user to add a card to their collection. By scanning the name and the unique card identifier of the card, it will send the information to the TCGDex API and check if the card exists.

4.2.3.1 Screen Images



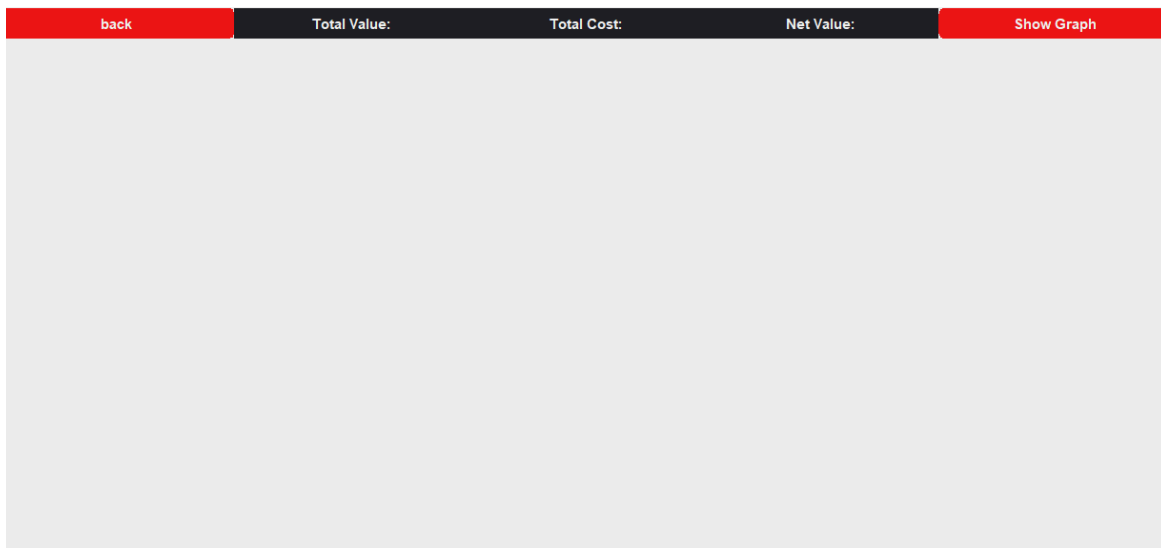
4.2.3.2 Objects and Actions

The objects of the page are the back button, which brings you back to the main menu, and the camera object that tries to scan for a pokemon card.

4.2.5 Collection Page

The collection page contains the set of all the cards in the user's collection. It also shows information about the totality of the collection such as: the total value of the cards, the total cost of taking all the cards, and the net worth of the collection so the user can keep track of how much they're losing or gaining.

4.2.4.1 Screen Images



4.2.4.2 Objects and Actions

The objects of this page will be all the images of the card collection (empty for the example page above), a page scroll that appears when the user has enough cards on their page, a button to go back to the main menu. The Show Graph button shows the change in price over time, prices on the TCG Market change frequently, so having a graph to show how the value of your card has changed over time would be useful.

5 Restrictions, limitations, and constraints

- Must be written in java
- The tool must operate in a Microsoft environment.
- Must work in IntelliJ.
- The information store will be an SQL database

6 Testing Issues (SLO #2.v)

6.1 Types of tests

- (1). 3 accuracy and 1 performance, and one user interface, and one repeatability

6.2 List of Test Cases

You should document each test case in the following format:

Test Type	Accuracy Test
Testing range	User login feature
Testing Input	User ID and Wrong password
Testing procedure	Enter user ID and wrong password Click Login
Expected Test Result	Unable to log in
Testers	Aidan Rodriguez
Test result	Passed

Test Type	Accuracy Test
Testing range	User login feature
Testing Input	User ID and Right password
Testing procedure	Enter user ID and right password Click Login
Expected Test Result	Logged in
Testers	Aidan Rodriguez
Test result	Passed

Test Type	Performance Test
Testing range	User Login sign-in Time less than 10 seconds
Testing Input	User Profile with Cards
Testing procedure	Enter user ID and User Password of a user with cards in database
Expected Test Result	Goes to main Menu
Testers	Aidan Rodriguez
Test result	Passed

Test Type	User Interface Test
Testing range	User Ability to Access all features from Main Menu UI
Testing Input	User Button Action
Testing procedure	Click Every button in the Main Menu, Be able to Go back to the Main Menu from the that Feature
Expected Test Result	Return back to Main Menu from any feature
Testers	Aidan Rodriguez
Test result	Passed

Test Type	Repeatability Test
Testing range	User can go to collection and get the same collection everytime
Testing Input	User cards added, User Button Action
Testing procedure	User adds cards, then exits and enters the collection multiple times
Expected Test Result	The same card output every enter
Testers	Aidan Rodriguez
Test result	Passed

6.3 Test Coverage

Functional

As a user I want to be able to look up a card and add it to my library and be able to find it by name. This is successful.

As a user i want the system to tell me the total value of my collection, this is a success

As a user i want to be able to order my library alphabetically or by price of card, this is failed.

As a user i want to be told if the card i am adding is a duplicate this is a success.

As a user I want to be able to delete cards from my collection, this is a success.

As a user I want the system to change the total value when a card is deleted, this is a success.

Nonfunctional

As a user i want the system to return the information in 5 seconds or less, this is a success.

As a user i want to be able to resize my library to see more of my collection at one time this is a success.

As a user i want to be able to move my card to see it from different angles. This is a fail

As a user i want to be able to use the camera of laptops to scan the cards. (before hand we use manual input) this is a success.

As a user i want the buttons to change color when i hover the mouse over them, this is a success.

As a user I want the text inputs to clear their default text when clicked, this is a success.

7 Appendices

7.1 Packaging and installation issues

How to install and prepare the system to run

1. Must have Windows 11
2. Must have Windows Key
3. Must have working Computer
4. Must have intellij Installed
5. Access to Github
6. Download [tesseract-ocr-w64-setup-5.5.0.20241111.exe](#) from <https://github.com/UB-Mannheim/tesseract/wiki>, and set it up
7. Download OpenCV 4.9.0 <https://opencv.org/releases/> for windows and set it up
8. Press Window + R
9. Type “sysdm.cpl” in the prompt
10. Go Advanced
11. Click Environment Variables
12. Create a new Path Variable
 - a. Variable Name: TESSDATA_PREFIX
 - b. Variable Value: C:\...\Tesseract-OCR\
13. Open Project on IntelliJ
14. Click File + Project Structure
15. Click Dependencies
16. Click “Maven: org.openpnp:opencv:4.9.0-0” dependency
17. Press add and Native Library C:\...\opencv\build\java\x64
18. Click Run + Edit Configuration on IntelliJ
19. Click Modify Options + VM Options
20. Add `--enable-native-access=ALL-UNNAMED` to the first prompt
21. Run Program

7.2 User Manual

Create account:

Create an account to be able to use PokeBin.

1. Launch the application.
2. If you do not have an account set up, select the option “New Account”
3. Enter your desired username and password.
4. Select “Confirm”, you are now able to login using those credentials.
5. Select “Back” if you don’t want to create a new account.

Login:

Login to view your collection and add cards.

1. Launch the application.
2. If you have an account set up, start entering your information.
3. Enter your username and password.
4. Select login, you are now greeted with the home menu.

Manual Input:

Add cards to your collection.

1. Upon logging in, and viewing the homescreen, select manual input.
2. There are two options to fill, one being the name of the pokemon on the card, and the second one being the ID of the card located on the bottom.
3. Enter the Pokemon’s name, please exclude any attached variants. For example, EX, V, Radiant, etc..
4. Enter the Pokemon card’s ID. This will be located on the bottom left or right, and it is usually 3 digits followed by a “/” and 3 more digits, only enter the first 3 digits.
5. Select “Add Card” once finished.
6. Select “Back” if you don’t want to add a card anymore.
7. If selected “Add Card”, a popup will appear showing the cards information.
8. If the card is correct and you want to add it to your collection, select “Yes”.
9. If the card isn’t the one you want to add, select “No”.
10. If the card you entered doesn’t exist, an error will appear, select “Back”.

Scan Card:

Add cards to your collection.

1. Upon logging in, and viewing the homescreen, select “scan card”.
2. A camera will appear, place the card you desire to add towards the middle of the camera, so that the whole card is showing.
3. Select “Scan” at the bottom to take a snapshot of the card, which will then prompt a popup.
4. Select “Back” at the top if you wish to exit the scan card section.
5. Upon scanning, the popup will show you the card's information.
6. If the card is correct and you want to add it to your collection, select “Yes”.
7. If the card isn’t the one you want to add, select “No”.

8. If the card you entered doesn't exist, an error will appear, select "Back".

View Collection:

View all cards you added to your collection.

1. Upon logging in, and viewing the homescreen, select "Collection".
2. You will be able to see all of the cards you have added to your collection.
3. At the top right you can see the total price combined of all the cards you have in your collection.
4. You can select each card individually which will display a popup.
5. This popup contains all of the card's information.
6. You can select "Back" to go back to viewing the collection.
7. You can select "Delete Card?" to delete the card from your collection.

7.3 Open Issues

We thought of implementing the ability to create binders, which are similar to the collection system, but can be customizable and you can have multiple. Unfortunately we ran out of time and never got to the implementation process.

Our camera is something we really wanted to get working 100%, but there were a lot of issues we ran into. Just setting up the camera requires installation of many external files, as well as updating the system's PATH. The camera is able to recognize text on the screen, but when doing testing it is very inconsistent. We were able to make bandaid fixes, but ultimately never got to reach its full potential.

The API we use has given us several issues, the biggest being that some cards don't give the correct price or no price at all. The API we have chosen is an open source free to use API, so it makes sense that it's not perfect compared to something you would have to pay for. Fortunately, all other aspects like the card image, as well as card information are correct.

7.4 Lessons Learned

7.4.1 Project Management & Task Allocations (SLO #2.i)

Make sure your response covers the following aspects (refer to the Project Evaluation Rubrics to see how this part will be evaluated).

- How your team allocate tasks and responsibilities and where could you improve in the future?
Our team allocated responsibilities by dividing up tasks whether through someone focusing on the DataBase, passing data from the API, working on the GUI, and learning more implementation with Java and Swing.
- How does your team do project planning activities?
Every week we created objectives for us all to do keeping us in check. We would meet on a call every Sunday to plan what our other meetings will be during the school week.
- How much effort does your team perform risk analysis along the process?

We heavily performed risk analysis because of how many cards there are in Pokemon. So many constraints with so many characteristics. So we performed a lot of risk analysis.

- How did your team update each other's progress and adjust your plans?
We updated each other's progress by the end of the meeting collaborations. We tried to sort of implement everything every time to see how it would pair together along with keeping track of how completed parts were.
- How did your team track changes and respond to changes that occurred?

We definitely needed to pivot at times. Sometimes we felt like we were ahead and not worried but something happened with our manual input scan because there multiple cards with the same moves and we needed to find a way to respond, what we did was add an extra ID that the user inputted and that the API had to look extra for in order to get the correct card.

7.4.2 Implementation (SLO #2.iv)

Make sure your response covers the following aspects (refer to the Project Evaluation Rubrics to see how this part will be evaluated).

- How did your team perform code review and refactoring to improve code quality? Provide a few code snippets (screenshots) that can best represent the team's coding quality.

```
private HashMap<String, FullCard> collectionCards = new HashMap<String, FullCard>(); //holds the cards 4 usages

public Color maincolor = new Color( r: 240, g: 240, b: 240); 23 usages
public Color coloraccent1 = new Color( r: 238, g: 21, b: 21); 14 usages
public Color coloraccent2 = new Color( r: 34, g: 34, b: 36); 14 usages
public Database db = new Database(); 7 usages
public int userID = 0; 2 usages

LoginUI loginPanel; 2 usages
MainMenuUI mainMenuPanel; 2 usages
CollectionUI collectionPanel; 3 usages
ManualInputUI manualInputPanel; 2 usages
ScanUI scanCardPanel; 2 usages

public MainFrame() { 1 usage  AverageRiceEnjoyer1

    //database part
    try
    {
        db.loadUsers( filename: "src/main/java/org/example/users");
        db.loadCards( filename: "src/main/java/org/example/Cards");
        db.loadCollections( filename: "src/main/java/org/example/card holders");
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}
```

```
//takes the panel name as input, each panel needs to go through the frame object parameter to change, slightly faster
public void changePanel(String name) 8 usages  ⚡ AverageRiceEnjoyer1 *
{
    CardLayout cl = (CardLayout) mpnl.getLayout();
    cl.show(mpnl, String.valueOf(name));
}

//adds FullCard argument to CollectionCards arraylist, local storage
public void addToCollect(FullCard tempcard) 2 usages  ⚡ AverageRiceEnjoyer1
{
    if(!collectionNums.containsKey(tempcard.getHashCode()))
    {
        collectionNums.put(tempcard.getHashCode(), 1);
        collectionCards.put(tempcard.getHashCode(), tempcard);
    }
    else
    {
        collectionNums.replace(tempcard.getHashCode(), collectionNums.get(tempcard.getHashCode()) + 1);
    }
}
```

- How much is your implementation consistent with your system design?

Yes, we definitely implemented a DAO, multiple interfaces, and an MVC, nothing we made was a facade, originally we were going to implement one but later implementation saw the design become less useful for the project.

7.4.3 Design Patterns

What are the design patterns used and why?

Interface, MVC pattern, and DAO pattern were used in our project. The interface was used since we were dealing with a lot of pages for the classes. MVC pattern was used since we had a lot of behind the scenes code like calculating totals then sending it out to show as a popup on the screen. We also used the DAO pattern since we were working with an API and needed the class to follow the CRUD standard which is important because the card class is changed a lot, whether it be adding to it or updating it, etc..

7.4.4 Team Communications

How team communications were conducted and where could you improve in the future?

Team communications were conducted via discord when we were NOT at school and provided a plan for upcoming weeks, usually here instead of working on it we would discuss issues we are facing or some positives that we have done. Aside from the Online meetings we would meet throughout the school week primarily on Tuesdays and Thursdays with for hours implementing what we discussed along with improvements we felt like contributed better.

7.4.4 Technologies Practiced (SLO #7)

Describe all new technologies/skills you would not have used/learned if you were not given the opportunity to work on the course project.

We were all very new to Java, and most of us didn't have an interest in learning it. GUI design was also a big learning curve, using swing. Working with the camera and seeing how it would read text was something new to us as well.

7.4.6 Software Security Practice (SLO #4.iii)

If given extra time, how would your team have incorporated software security into the design and implementation of the software system?

If we needed to implement some software security in the design we all agreed that we needed to add censoring to the passwords of users, if we really wanted to we could add a 2 step authentication code. Another idea that we could have implemented was the forgot password feature which would need to email the user, meaning we would need to add the email to the users database.

- Identify specific design or implementation improvements you would have made to enhance security without compromising user experience or efficiency.

When the user creates an account we would also prompt the user for their emails so in case of password forgetting or they want to change password it would send an email to the user. Doing this we would need to update our design for the login/create account page, along with adding the email to the database.

- Discuss any security mechanisms or practices you would apply if there was more time (e.g., authentication, access control, secure data handling, or encryption).

As stated earlier we would implement two factor authentication and censoring the password for the users security. All in all our project doesn't really have that much confidentiality besides access to pokemon cards and how much the user had in their collection. If potential hackers came in they would only be able to view the collection but they wouldn't be able to actually have the physical cards since it's all virtual.

7.4.5 Desirable Changes

Assuming that you have another month to work on the project, what aspects of the system you would like to improve? What are the additional features you want to add to the system? [Each student should use a separate paragraph to respond to the questions]

Lucas- If we could add any additional implementations if we had one more month to work, I'd add more security. Currently if you input the password there isn't any security like authentication code or even just censoring the password. I feel like a better camera OR even moving towards mobile app would be better as well.

Looking at our system, what I would like to improve is the database system. If I had more time I would improve the entry and saving of cards and items into the database. I would also like to change from a

json based system to a Sql based system. A new feature that i want to add is that you can add cards in your collections to a deck and then be able to test the deck in a testing ground. -tyler

Frankie- If we had another month to work on the project, I would have liked to get the camera fully functioning and polished so we didn't have to externally download files. Currently it has trouble reading the card fully and presenting the right one. Some features I would like added is a binder system, to add more creativity for the user.

Aidan - If I had an extra month I would have made the scanner work better, it works fine now, but it definitely could use additional functionality like cases where the card ID is in a different part of the card or the lighting isn't right or any other problems. At least the camera works though, so I really can't complain.

7.4.6 Challenges Faced

Among requirements specification, system design, and system implementation, which one you think is the hardest task? Why? [Each student should use a separate paragraph to respond to the questions]

Lucas - Some challenges faced for our project I feel like everyone had different challenges. I feel like our greatest challenge was that for some reason the API didn't have specific prices. For some reason, we didn't know how to overcome it because if it didn't have a price the card wouldn't come up as an image. TO overcome this we needed to find a way to get the required specification to do this we made it show N/A to actually get the car image and store into the personal collection

I would say that requirements specification was the hardest task. This is because it has the most variance for me in that it's hard to get fully into someone else's head and view our project objectively. This means that as we went forward we would run into things that we were unsure if it was an actual requirement or something that would be obvious to a user. Since our project was so simple we often ran into the problem of not believing that we were all done with our requirements. -tyler

Frankie- I'm split between requirement specifications and system implementation. We never really struggled with how we wanted to design the app, we had a pretty good idea of the theme and looks and features. The requirement specifications were hard since as we started implementing code, we would run into issues where we would need to go back and change the requirements. At first as well it was difficult to understand what the requirements should be until we started diving into the implementation.

Aidan - The hardest task was timing. Often we would push off coding for a while until we all had the time, but none of us ever really had time. The design wasn't too bad, we knew pretty early what we wanted the app to look like and how it would feel from a user's perspective, but actually making it look that way with Java Swing of all things and having it interface with a bunch of different java libraries we've never seen before was a challenge compounded by the lack of time.